System Programming 2021: Vaccinated HW1

1. Problem Description

Due to the coronavirus, people are now strongly recommended to be vaccinated. To make the process of vaccination go smoothly, you are expected to implement a simplified COVID-19 vaccination registration system: csieVaccination .
The csieVaccination system is comprised of read and write servers, both can access a file registerRecord that records information of user's registration. When a server gets requests from clients, it will respond according to the content of the file. A read server can tell the client which brand of vaccine they registered before. A write server can modify the file to record the registration.
You are expected to complete the following tasks:

1. Implement the servers. The provided sample source code server.c can be compiled as simple read/write servers so you don't have to code them from scratch. Details will be described in the later part.

2. Modify the code in order that the servers will not be blocked by any single request, but can deal with many requests simultaneously. You might want to use `select()` to implement the multiplexing system. However, remember not to do anything that will result in busy waiting.

3. Guarantee the correctness of file content when it is being modified. You might want to use file lock to protect the file.

2. Running the Sample Servers

The provided sample code can be compiled as simple servers.
Compile
You should write your own makefile to compile the code. You can use the provided command to produce execution files first.

```
$ gcc server.c -D WRITE_SERVER -o write_server
$ gcc server.c -D READ_SERVER -o read_server
```

Run

The server will be listening to {port}

```
$ ./write_server {port}
$ ./read_server {port}
```

The sample server handled the part of socket programming, so you are able to connect to the sample server once you run it. Your client and server can communicate to each other.
Feel free to modify any part of the code as you need, or implement your own server.

3. Testing your Servers at Client Side

`telnet {hostname} {port}`

If your server is running on linux1.csie.ntu.edu.tw (http://linux1.csie.ntu.edu.tw), and listening to port 7777, your command on the client side will be

`telnet linux1.csie.ntu.edu.tw 7777`

If you are running sample servers, try to type something on the client side. You will see some messages from servers.

4. About the Registration Record File

The servers should be able to access the file registerRecord. The file includes 20 user's information made up with a user id (range from 902001 to 902020) and user's preference for vaccine. A user can determine their preference for different brands of vaccine. The variable `AZ`, `BNT` and `Moderna` should be set as their preference order(ascending order from 1). For an instance, Stanley's preference for vaccine is `Moderna > AZ > BNT`, his registerRecord will be

- `AZ` = 2
- `BNT` = 3
- `Moderna` = 1

In addition, the preference order should not repeat nor coflict, such as

- `AZ` = 1
- `BNT` = 2
- `Moderna` = 1

```
typedef struct {
 int id;          //902001-902020
 int AZ;
 int BNT;
 int Moderna;
} registerRecord;
```

For those who are not familiar with c language, you might want to check this website (https://www.geeksforgeeks.org/readwrite-structure-file-c/).
Noted that there is registerRecord released for testing, however your code should not depend on this version of registerRecord. That is, there is possibility of judging your homework with other versions of registerRecord.

5. Sample input and output

Read Server

A client can check their preference order. Once it connect to a read server, the terminal will show the following:
`Please enter your id (to check your preference order):`
If you type a id (for example, `902001`) on the client side, the server shall reply:
`Your preference order is Moderna > BNT > AZ.`
and close the connection from client.
But, if someone else is ordering using the same id, the server shall reply:
`Locked.`
and close the connection from client.

Write Server

A user can make preference record. It will first show how a user's previous preference order just like a read server, then ask for new preference order.
Once it connect to a write server, the terminal will show the following
`Please enter your id (to check your preference order):`
If you type an id (for example, `902001`) on the client side, the server shall reply his/her preference order, following by a prompt on the nextline

```
Your preference order is Moderna > BNT > AZ.
Please input your preference order respectively(AZ,BNT,Moderna):
```

If you type a command (for example, `1 2 3`), the server shall modify the registerRecord file and reply:

`Preference order for 902001 modified successed, new preference order is AZ > BNT > Moderna.`

and close the connection from client.

But, if someone else is using the same id, the server shall reply:

`Locked.`

and close the connection from client.

It is your responsibility to check the above rules! If the user's input violates the rules, it will be seen as a failed operation, which will print out

`[Error] Operation failed. Please try again.`

and close the connection from client.

6. Grading

1. Finish the Makefile. (0.5 points)
    - The servers, named exactly `read_server` and `write_server`, can be produced and perform successfully by `make` command.
2. A read server handles requests correctly. (0.5 point)
    - There would be only one request at a time for the subtask.
3. A write server handles requests correctly. (1 point)
    - There would be only one request at a time for the subtask.
4. Multiple requests issued to one read server. (1 point)
    - Eg. Two requests issued to one read server and access the same ID content should be run successfully.
    - There would be 10 requests at most in the mean time issued to one read server.
5. Multiple requests issued to a write server. (1.5 points)
    - Eg. Two requests issued to one write server and access the same ID content should trigger "Locked.". However, if they access different ID content, they should run successfully.
    - There would be 10 requests at most in the mean time issued to one write server.

6. Requests issued to write servers (1 point)
    - We might issue serveral requests to multiple write servers to check if you implement the file lock part.
    - There would be 10 requests issued to 4 write servers at most in the mean time .
7. Requests issued to read servers and write servers (1.5 points)
    - In this part, there will be no limitation. Thus, you might want to think about any possible test data.
    - There would be 20 requests issued to 10 servers at most in the mean time .
8. Report(1 point)
    Last of the all, you have to hand in a report. Noted that “busy waiting” should not happen on any servers, therefore, you will get 0.5 point deduction for being busy waiting.

For every task, you have to pass all the testdata to receive point for that part. So make sure that you think twice during coding.

7. Submission

Your assignment should be submitted to github before deadline. The submisstion should include at least two files:

1. Makefile
2. server.c

You can submit other .c, .h files, as long as they can be compiled to two executable files named `read_server` and `write_server` with Makefile.
However, do not submit files generated by Makefile. You should `make clean` before you submit.
You are encouraged to submit a readme.txt, where you can briefly state how do you finish your program and something valuable you want to explain.

8. Reminder

1. Plagiarism is STRICTLY prohibited.
2. Your credits will be deducted 10% for each day delay. A late submission is better than absence.
3. If you have any question, feel free to contact us via email ntucsiesp@gmail.com (mailto:ntucsiesp@gmail.com) or come during TA hours.

4. Any form of cheating, lying, or plagiarism will not be tolerated. Students can get zero scores or fail the class for those kinds of misconducts.
5. Please start your work as soon as possible, do NOT leave it until the last day!
6. Watch out for the typos!